

Technical Summary:

Dynamic Voting for Efficient Reasoning in Large Language Models

Kai-Yu Lu

1 Research Problem and Motivation

Large language models often solve reasoning problems by producing an explicit step by step rationale, commonly referred to as *chain of thought* (CoT). While CoT improves reasoning over direct answering, individual reasoning traces frequently contain factual errors and hallucinated steps, which leads to incorrect final answers. Multi path voting techniques, most notably *self consistency* (SC), mitigate this issue by sampling many diverse CoT reasoning paths and aggregating their final answers via majority vote.

The central limitation is computational cost. Self consistency typically samples up to forty reasoning paths per question, and other voting variants sample even more, which multiplies inference cost by tens of times. Empirical evidence in the paper indicates substantial resource waste: on SVAMP, 57.5% of problems reach 95.7% accuracy with only three unanimous reasoning paths, and generating additional paths for these problems yields negligible gains. This motivates an adaptive allocation of sampling budget, where easy questions exit early and difficult questions consume more paths.

The research problem is therefore to retain the accuracy benefits of multi path voting while substantially reducing the number of sampled reasoning paths.

2 Related Work

2.1 Chain of Thought Prompting

Chain of thought prompting elicits intermediate reasoning steps that improve performance on reasoning heavy tasks, including mathematics and commonsense question answering. The method remains vulnerable to erroneous intermediate steps that propagate to final answers.

2.2 Multi Path Voting

Self consistency and related approaches sample multiple reasoning traces under stochastic decoding and aggregate answers, typically through majority vote. These methods improve robustness but incur significant inference cost due to repeated generation. Some calibration approaches introduce auxiliary verifiers to rank or filter rationales, which increases system complexity and can degrade under distribution shift.

2.3 Early Exiting

Early exiting methods reduce computation by terminating inference once sufficient confidence is reached. Prior work applies early exiting to feature cascades, retrieval pipelines, and deep neural networks using confidence indicators from intermediate states. The paper adapts the early exit principle to multi path voting for reasoning, using a voting based confidence signal.

Table 1: Datasets, task types, and evaluation splits used in the paper.

Dataset	Task Type	Split Used	Construction and Notes
GSM8K	Arithmetic reasoning	Test	Standard test split used for evaluation.
SVAMP	Arithmetic reasoning	Test	Standard test split used for evaluation.
CSQA	Commonsense QA	Dev	Dev split used because test labels are not available.
StrategyQA	Commonsense QA	Sampled subset	1000 questions randomly sampled from the train filtered set to avoid leaderboard submission constraints.
LetterConcat	Symbolic reasoning	Constructed test set	1000 examples generated by combining the 1000 most common surnames and first names from Name Census, then applying the LetterConcat procedure.

3 Dataset Construction

The paper evaluates Dynamic Voting on five representative reasoning benchmarks selected to be moderately challenging for the chosen model.

No new annotation protocol is introduced for the existing datasets. Dataset construction primarily concerns the controlled creation of evaluation sets for LetterConcat and StrategyQA to enable repeated threshold sweeps without external submission constraints.

4 Query Protocol and Task Definitions

4.1 Reasoning Path and Voting

A *reasoning path* is a single sampled model completion that includes an intermediate rationale and a final answer. A *vote* is the final answer extracted from a reasoning path. Multi path voting aggregates the answers from multiple paths using majority voting to produce the final prediction.

4.2 Voting Round and Stopping Criteria

Dynamic Voting proceeds in rounds. In each round, a set of reasoning paths is available, votes are computed, and a consistency based confidence criterion is evaluated. If the criterion is satisfied, the procedure exits early and returns the current majority answer. Otherwise, an additional reasoning path is generated and included in the next voting round.

4.3 Path Usage Constraints

The paper defines the maximum voting budget as forty votes per question, aligned with the standard self consistency setting. Path usage constraints are reported as $\leq 25\%$, $\leq 50\%$, $\leq 75\%$, and $\leq 100\%$, where 100% corresponds to forty votes. For self consistency, a constraint directly determines a reduced fixed number of votes per question. For Dynamic Voting, the constraint refers to the average percentage of used paths on a dataset relative to the forty vote maximum, because the number of sampled paths varies by question.

4.4 Voting Consistency as Confidence

The paper quantifies confidence via voting consistency, defined as the proportion of votes supporting the current majority answer in a given round.

$$C = \frac{\max_{a \in \mathcal{A}} \text{count}(a)}{|\mathcal{A}|} \quad (1)$$

Symbol definitions and explanation.

- $\mathcal{A}$ denotes the multiset of voted final answers produced by the currently generated reasoning paths in the current voting round.
- $|\mathcal{A}|$ is the total number of votes in the round, equal to the number of generated reasoning paths used for voting in that round.
- a denotes a candidate final answer string after answer extraction.
- $\text{count}(a)$ counts how many paths output answer a in the current round.
- $\max_{a \in \mathcal{A}} \text{count}(a)$ is the number of votes for the majority answer.

Mathematical background. Equation (1) operationalizes a majority concentration statistic. It maps a set of categorical outcomes into a scalar in $[0, 1]$ that measures the dominance of the majority class.

Intuition. A higher C indicates that independently sampled reasoning paths converge on the same final answer, which the paper treats as higher confidence. For example, a round with two votes for the majority answer out of three total votes yields $C = 2/3 = 0.67$, while three out of four yields $C = 3/4 = 0.75$.

Role in the study. Equation (1) is the decision signal for early exiting in Dynamic Voting. Thresholds applied to C determine whether additional reasoning paths are generated.

5 Modeling Approach

5.1 Overall Dynamic Voting Process

Dynamic Voting is a multi round generate then vote procedure with early exiting based on consistency. The method is parameterized by:

- Consistency threshold t : the minimum consistency required to stop voting and output the current result.
- Initial number of votes v : the number of reasoning paths generated in the first round.
- Maximum number of votes V : the upper bound on total generated paths for a question.
- Problem instance p : an input question from a dataset.

The operational steps are:

1. Initialize: set t , v , and V .
2. Round 1 generation: sample v reasoning paths for problem p using stochastic decoding.
3. Voting: extract final answers, compute majority vote, and compute consistency C using Equation (1).
4. Early exit check: if $C \geq t$, terminate and return the current majority answer.

5. Otherwise: sample one additional reasoning path, add it to the pool, and repeat voting and the exit check.
6. Termination: stop when $C \geq t$ or the total number of sampled paths reaches V .

This design reallocates sampling budget across questions. Questions that rapidly reach high consistency exit early, while questions with persistent disagreement continue to sample more paths until a threshold is met or the budget is exhausted.

5.2 Confidence Based Dynamic Voting

Confidence based Dynamic Voting (CDV) applies a fixed consistency threshold t_c for all voting rounds. Early exiting occurs once the proportion of majority votes in the current round reaches t_c . The paper highlights that a constant threshold is used to avoid fine grained per round tuning, even though later rounds typically correspond to harder questions that are less likely to reach strict consistency.

5.3 Percentage Based Dynamic Voting

Percentage based Dynamic Voting (PDV) targets batch evaluation. Instead of using a fixed numeric threshold, PDV sets a percentage threshold t_p . In each round, questions are ranked by current round consistency, and the top $t_p\%$ of questions exit early. This strategy avoids requiring prior knowledge about the model or question difficulty when selecting a numeric threshold. The paper notes that PDV is less suitable for single question usage because the decision rule is defined over a batch.

6 Empirical Results

6.1 Experimental Setup

Model. The experiments use the GPT 3.5 turbo 0301 API.

Prompting regimes. Two settings are evaluated:

- Few shot: demonstrations follow the self consistency setup, with minor modifications to form dialog histories compatible with the API interface.
- Zero shot: a single sentence instruction constrains response length and enforces a standardized final answer format.

Decoding for diversity. Temperature sampling is used to generate diverse reasoning paths, with temperature set to 1.5 and no top k or top p truncation.

Evaluation metric. The reported metric is accuracy, defined as the percentage of questions for which the final predicted answer matches the gold answer under the dataset specific evaluation.

6.2 Threshold Selection and Hyperparameters

Thresholds are selected using a validation set of 100 randomly sampled training examples per dataset. The search procedure generates forty paths once for those examples, and then evaluates candidate thresholds.

Table 2: Key hyperparameters and threshold search configuration.

Component	Configuration
Maximum vote budget	Forty votes per question defines 100% path usage.
Initial votes in Dynamic Voting	Three votes in the first round.
Path usage constraints	$\leq 25\%$, $\leq 50\%$, $\leq 75\%$, $\leq 100\%$ relative to forty votes.
CDV threshold search	$t_c \in [0.10, 0.95]$ with step size 0.05.
PDV threshold search	$t_p \in \{1, 2, \dots, 30\}$ with step size 1.
Validation set size	100 examples randomly sampled from the training split per dataset.
Decoding	Temperature = 1.5, no top k, no top p truncation.

Table 3: Accuracies of self consistency (SC) and Dynamic Voting under different path usage constraints. Bracketed values are actual average path usage on the evaluation split (%).

Task	Method	Few shot accuracy (usage %)				Zero shot accuracy (usage %)			
		$\leq 25\%$	$\leq 50\%$	$\leq 75\%$	$\leq 100\%$	$\leq 25\%$	$\leq 50\%$	$\leq 75\%$	$\leq 100\%$
GSM8K	SC	82.6 (25.0)	86.1 (50.0)	87.3 (75.0)	87.9 (100.0)	80.2 (25.0)	84.0 (50.0)	85.1 (75.0)	85.4 (100.0)
	CDV	86.5 (25.0)	87.9 (42.9)	87.9 (52.3)	87.9 (52.3)	82.7 (28.4)	85.1 (46.6)	85.3 (60.4)	85.4 (68.0)
	PDV	85.8 (24.7)	87.6 (43.1)	87.9 (65.5)	87.9 (85.9)	84.2 (25.6)	85.4 (43.1)	85.7 (58.1)	85.4 (85.9)
SVAMP	SC	84.5 (25.0)	86.3 (50.0)	86.3 (75.0)	86.7 (100.0)	82.9 (25.0)	85.9 (50.0)	87.6 (75.0)	87.4 (100.0)
	CDV	86.0 (23.6)	86.1 (31.4)	86.1 (37.5)	86.1 (37.5)	85.1 (23.8)	87.1 (40.7)	87.2 (57.0)	87.2 (57.0)
	PDV	86.2 (25.6)	86.8 (47.2)	86.7 (58.1)	86.7 (85.9)	86.4 (24.7)	87.8 (43.2)	87.4 (65.5)	87.4 (85.9)
CSQA	SC	80.1 (25.0)	80.5 (50.0)	80.6 (75.0)	81.0 (100.0)	72.2 (25.0)	75.8 (50.0)	75.6 (75.0)	75.8 (100.0)
	CDV	80.5 (23.3)	80.8 (37.1)	80.8 (40.2)	80.8 (40.2)	74.0 (18.4)	75.8 (39.6)	75.9 (64.4)	75.7 (93.4)
	PDV	80.3 (21.9)	81.0 (47.1)	81.0 (65.5)	81.0 (85.9)	75.6 (22.5)	76.0 (43.1)	75.8 (65.5)	75.8 (85.9)
StrategyQA	SC	75.7 (25.0)	77.0 (50.0)	77.2 (75.0)	76.7 (100.0)	71.8 (25.0)	73.5 (50.0)	74.2 (75.0)	73.7 (100.0)
	CDV	76.3 (22.0)	76.9 (43.6)	76.9 (54.8)	76.9 (55.1)	73.7 (22.5)	73.6 (38.6)	73.8 (64.3)	73.8 (67.2)
	PDV	76.4 (22.5)	77.4 (36.9)	76.7 (65.5)	76.7 (85.9)	73.0 (22.5)	74.1 (47.2)	73.7 (58.1)	73.7 (85.9)
LetterConcat	SC	85.0 (25.0)	86.2 (50.0)	86.8 (75.0)	86.7 (100.0)	62.4 (25.0)	79.8 (50.0)	86.3 (75.0)	89.9 (100.0)
	CDV	86.4 (23.4)	86.4 (31.5)	86.4 (31.5)	86.4 (31.5)	78.7 (30.9)	84.8 (48.5)	89.7 (72.4)	89.9 (97.0)
	PDV	87.0 (24.7)	86.8 (43.2)	86.7 (65.5)	86.7 (85.9)	75.1 (24.7)	89.3 (47.2)	89.8 (65.5)	89.9 (85.9)
AVG	SC	81.6 (25.0)	83.3 (50.0)	83.7 (75.0)	83.9 (100.0)	74.2 (25.0)	79.9 (50.0)	81.7 (75.0)	82.4 (100.0)
	CDV	83.2 (23.5)	83.8 (37.6)	83.8 (43.7)	83.8 (43.7)	78.9 (24.8)	81.3 (42.9)	82.3 (63.5)	82.3 (76.7)
	PDV	83.2 (23.9)	84.0 (43.6)	83.9 (64.2)	83.9 (85.9)	79.0 (24.0)	82.4 (44.6)	82.4 (62.4)	82.3 (85.9)

6.3 Main Accuracy and Efficiency Findings

The paper reports that Dynamic Voting matches self consistency accuracy while using substantially fewer reasoning paths on average across five datasets. It also improves accuracy under strict path budgets because it concentrates additional sampling on harder questions rather than uniformly sampling a fixed number of paths per question.

Table 3 reproduces the main results reported under multiple path usage constraints. Values are accuracy, and bracketed values are the actual average path usage percentage measured on the evaluation split.

6.4 Interpretation of Numerical Results

Comparable accuracy with fewer paths. On average across the five datasets, Dynamic Voting reaches the SC accuracy with substantially less than full path usage. The reported average usage for CDV remains far below 100% even in the $\leq 100\%$ setting because strict thresholds can terminate early for many questions without accuracy loss exceeding 0.1 percentage points.

Advantages under strict budgets. Under the $\leq 25\%$ constraint, Dynamic Voting exceeds SC on every dataset in both few shot and zero shot settings. The paper reports average improvements of 1.6 points in few shot and 4.7 points in zero shot when both approaches are constrained to 25% path usage, reflecting the benefit of adaptive allocation.

Large gains on symbolic reasoning in zero shot. LetterConcat exhibits substantial sensitivity to the number of sampled paths under SC, with 62.4 accuracy at 25% and 89.9 at 100% in zero shot. Dynamic Voting markedly improves the low budget regime, with CDV reaching 78.7 and PDV reaching 75.1 under approximately 25% usage, and PDV reaching 89.3 under approximately 50% usage.

Strict threshold setting. The paper reports a strict configuration with $t_c = 0.95$ and $t_p = 1$ that demonstrates negligible average accuracy loss while sharply reducing average path usage in CDV.

7 Summary

Dynamic Voting introduces an early exit mechanism for multi path voting in large language model reasoning. The key methodological choice is to use voting consistency, defined as the proportion of majority votes, as a confidence indicator that determines whether further path generation is necessary. Two practical strategies are proposed: CDV, which uses a fixed consistency threshold per round, and PDV, which exits a fixed proportion of questions per round in batch settings.

Empirical evaluation on arithmetic, commonsense, and symbolic reasoning tasks demonstrates that Dynamic Voting preserves the accuracy of self consistency while substantially reducing the number of sampled paths. The method also improves performance under strict path budgets by reallocating computation toward difficult questions, and it exhibits threshold robustness and compatibility with diverse prompts and voting techniques as described in the paper.